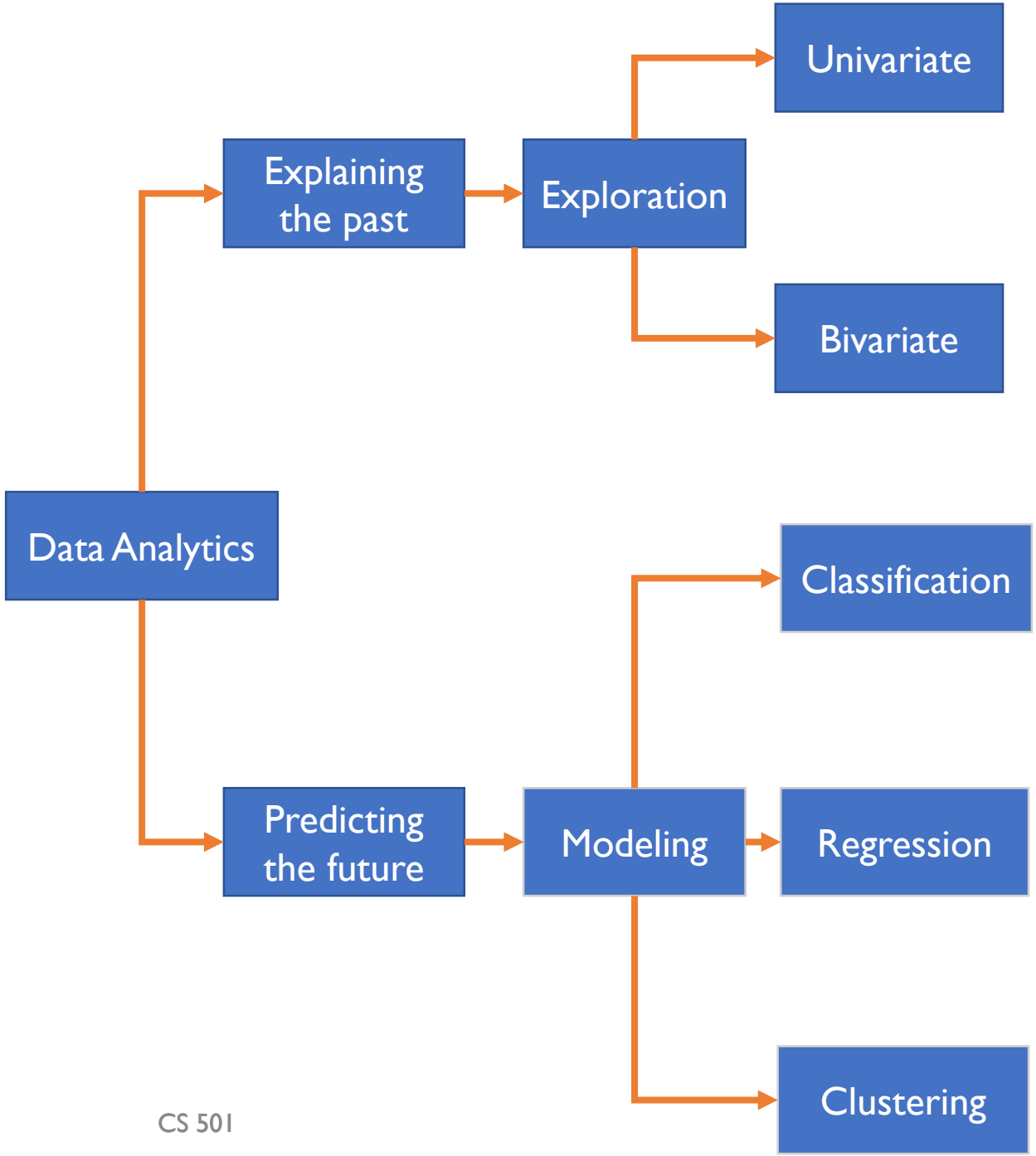
Clustering

CSE 451
Topics in Computer Science
Fall 2023
© Royal Military College of Canada


Data Analytics
Explaining the past
Exploration
Univariate
Bivariate
Predicting the future
Modeling
Classification
Regression
Clustering

Classical Machine Learning

Task Driven

Data Driven

Supervised Learning

(Pre Categorized Data)

- Classification
- Regression

Unsupervised Learning

(Unlabelled Data)

- Clustering
- Association
- Dimensionality Reduction

Output:

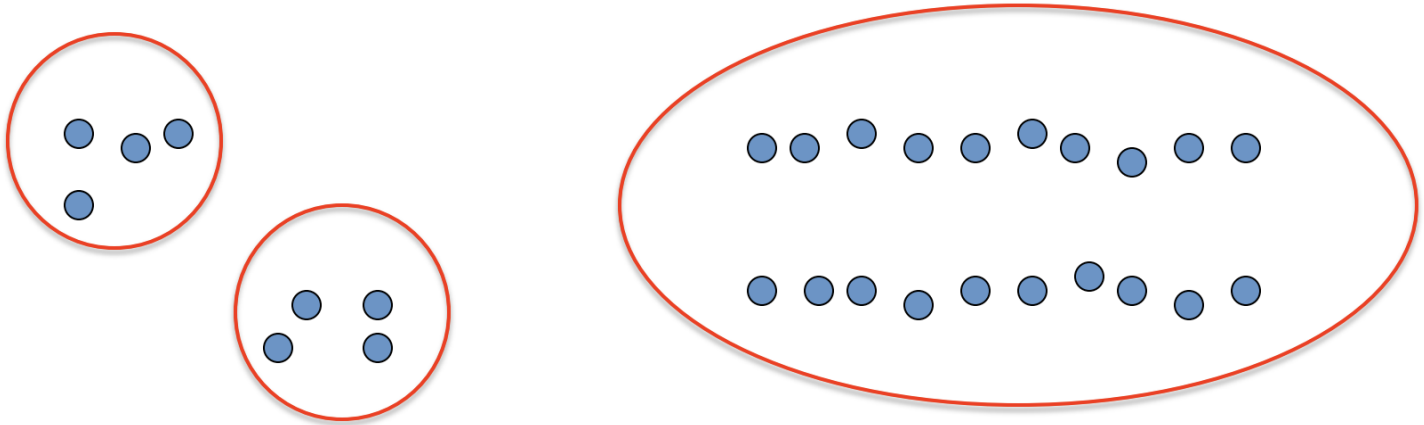
- Classification: A category
- Regression: A real-value
- Clustering: patterns within a group of uncategorized data
- Association: identify associations between different data objects
- Dimensionality Reduction: identify wider dependencies

Objective:

- Supervised Learning: Predictive analysis
- Unsupervised Learning: Pattern recognition

What is Clustering

- Clustering is a fundamental technique in *unsupervised* machine learning, which aims to group similar data points into clusters based on some similarity criteria.
- Clustering is a powerful technique in data science that allows us to discover patterns and structure in data <u>without prior knowledge</u> or <u>labels</u>.
- Example: 2D point patterns



Uses of Clustering

- customer segmentation in marketing
- image segmentation in computer vision
- gene expression analysis in bioinformatics, and
- anomaly detection in cybersecurity.

Example of use

Use of clustering for anomaly detection in network traffic analysis

- In this case, the network traffic is collected and analyzed to identify patterns of normal behavior.
- Clustering is then used to group similar traffic flows based on various features such as source IP address, destination IP address, protocol, port numbers, and packet sizes.
- Once the clusters are formed, the system can identify the typical behavior of each cluster and use it as a baseline for comparison with future traffic.
- If a new traffic flow is <u>significantly different</u> from the baseline behavior, it may be flagged as an anomaly or suspicious activity.
  - For example, a large amount of traffic with uncommon source IP addresses or using an unusual combination of protocols and ports may indicate a potential attack or breach attempt.

Types of clustering algorithms

There are many types of clustering algorithms, each with its own strengths and weaknesses.

1. Hierarchical clustering is a bottom-up approach that builds a tree-like structure called a dendrogram to represent the hierarchy of clusters.
2. Partitioning clustering is a top-down approach that assigns each data point to one of the pre-defined clusters based on some distance metric.
3. Density-based clustering is a non-parametric approach that identifies dense regions of data points separated by low-density regions.

Types of Clustering algorithms

Hierarchical

1. Agglomerative clustering
2. Divisive clustering
3. …..

Partitioning

1. K-means clustering
2. K-medoids clustering
3. …..

Density

1. DBSCAN
2. OPTICS
3. …..

Evaluation of Clustering

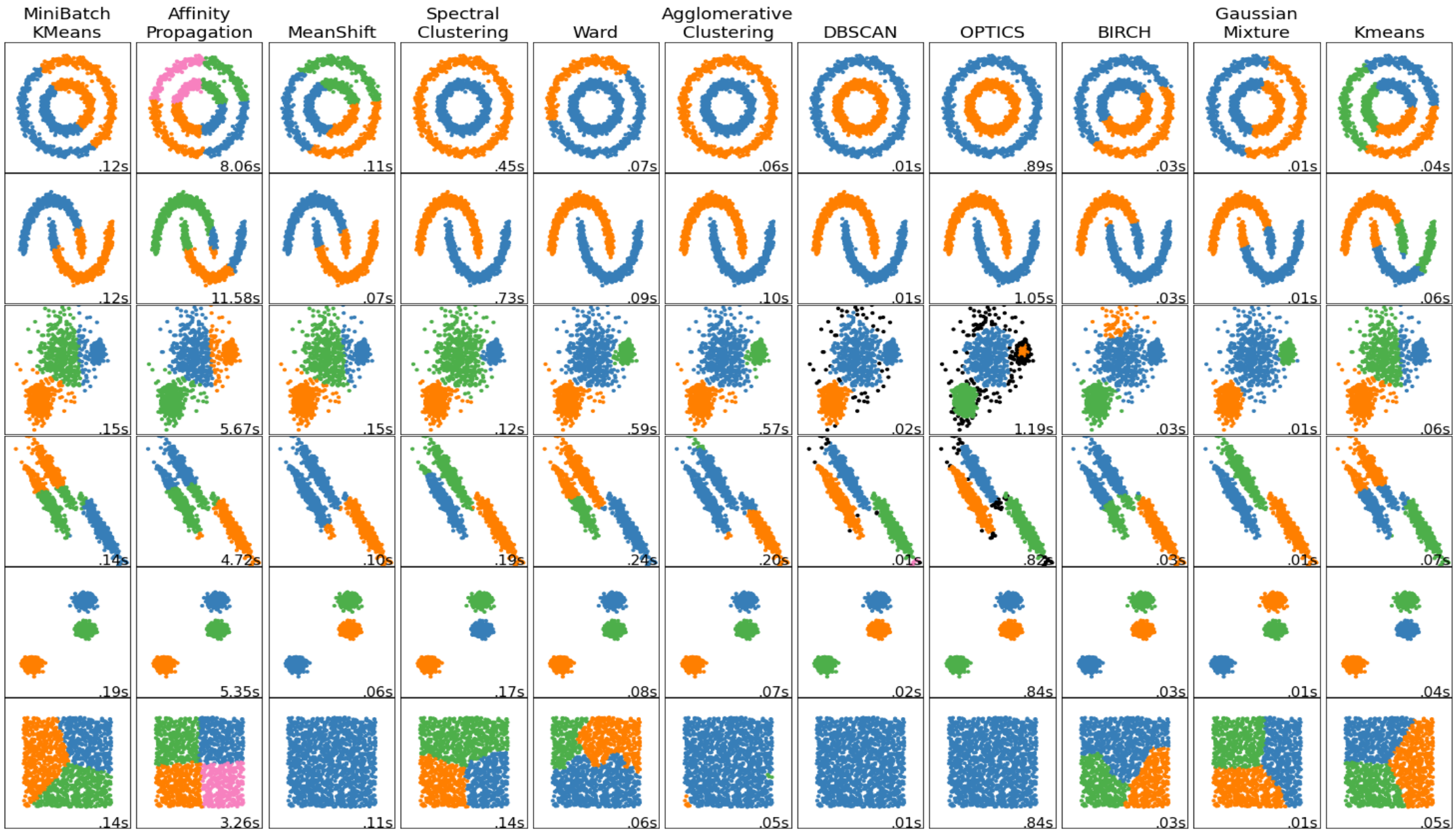
- Choosing the right clustering algorithm and tuning its parameters can be a challenging task, as it depends on the data characteristics, the clustering objectives, and the computational constraints.

- Therefore, evaluating the clustering results is also important to assess the quality of the clustering and compare different algorithms or parameters.

Goodness of Clustering

What constitutes good clustering?

- All clustering algorithms will produce clusters, regardless of whether the data contains them.
- There is no golden standard, **depends on goal**:
  - data reduction
  - "natural clusters"
  - "useful" clusters
  - outlier detection


MiniBatch KMeans
Affinity Propagation
MeanShift
Spectral Clustering
Ward
Agglomerative Clustering
DBSCAN
OPTICS
BIRCH
Gaussian Mixture
Kmeans
.12s 8.06s .11s .45s .07s .06s .01s .89s .03s .01s .04s
.12s 11.58s .07s .73s .09s .10s .01s 1.05s .03s .01s .06s
.15s 5.67s .15s .12s .59s .57s .02s 1.19s .03s .01s .06s
.14s 4.72s .10s .19s .24s .20s .01s .82s .03s .01s .07s
.19s 5.35s .06s .17s .08s .07s .02s .84s .03s .01s .04s
.14s 3.26s .11s .14s .06s .05s .01s .84s .03s .01s .05s

Hierarchical Clustering

Hierarchical Clustering

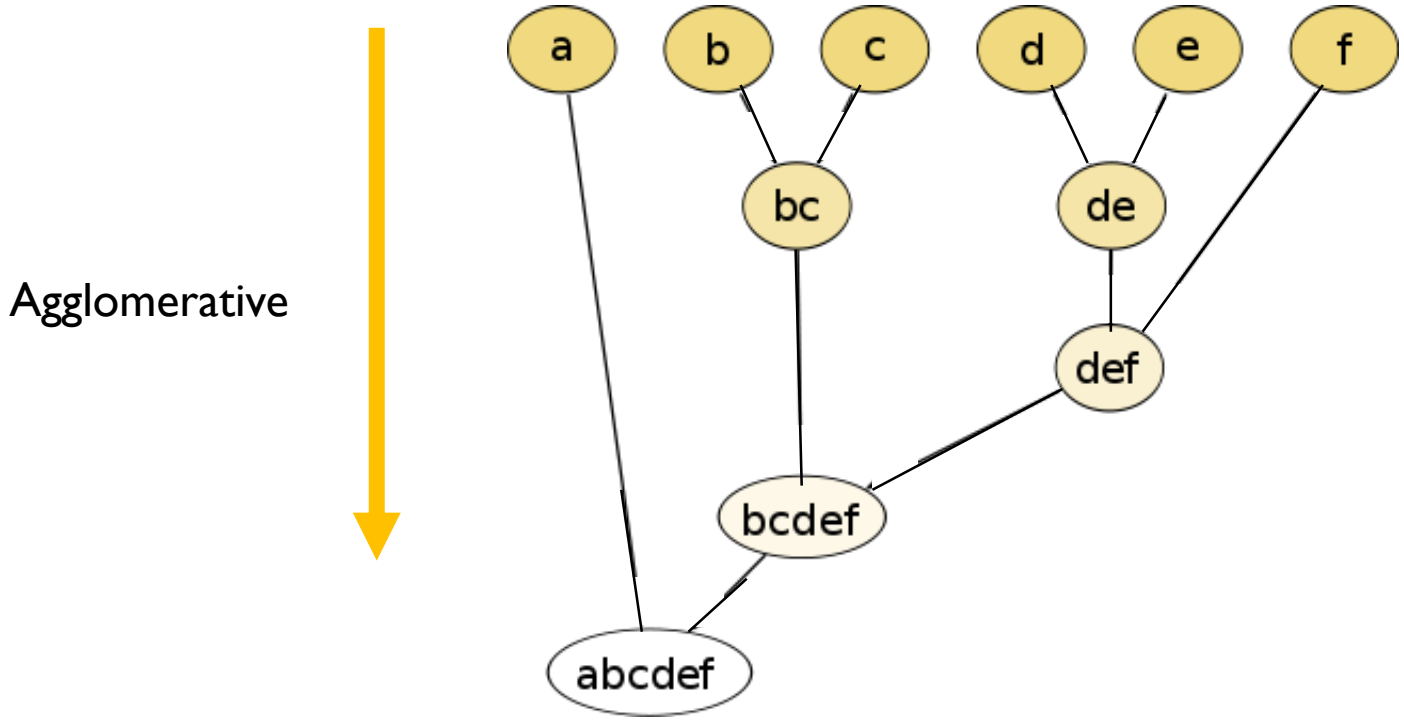
- a clustering method that creates a hierarchy of clusters, where each cluster is either a single data point or a combination of data points.

- The main advantage of hierarchical clustering is that it does not require the number of clusters to be pre-defined, unlike partitioning methods such as K-means.

- Instead, the number of clusters is determined by the user based on the dendrogram, which is a visual representation of the clustering hierarchy.

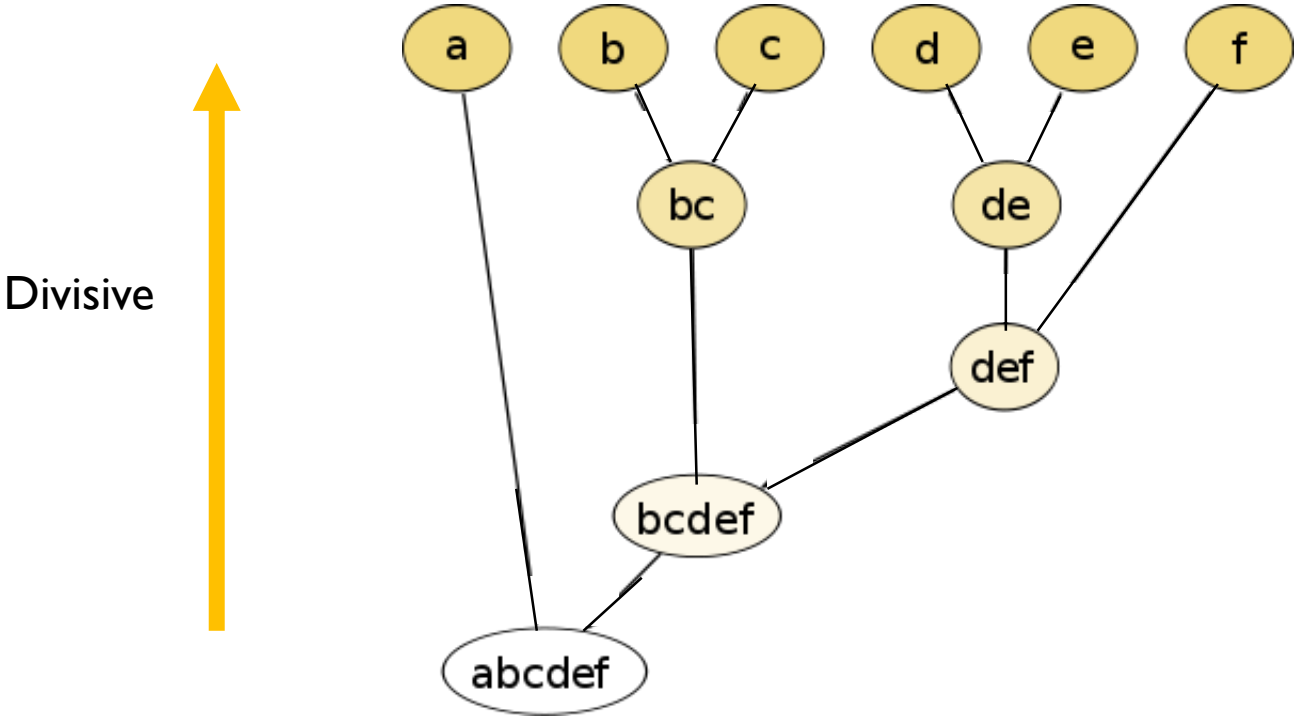
Hierarchical Clustering: Agglomerative

- Each data point starts as its own cluster, and …
- then the algorithm recursively merges the two closest clusters until a single cluster containing all the data points is formed.



Hierarchical Clustering: Divisive

- In divisive clustering, the process is reversed, starting with all data points in one cluster and …
- recursively dividing the clusters into smaller ones until each data point is its own cluster.



Hierarchical Clustering

- The dendrogram is a tree-like structure that shows the order in which the clusters were merged or divided, as well as their distances or similarities.

- But how to define distance/similarity?

Hierarchical Clustering

The distance between two clusters is typically defined by a linkage criterion, such as:

- Single linkage measures the minimum distance between any two points in the two clusters.
- Complete linkage measures the maximum distance between any two points in the two clusters.
- Average linkage measures the average distance between all pairs of points in the two clusters.

Example: Single linkage

Point	X	Y
A	1	1
B	2	2
C	5	5
D	6	6

- To use single linkage to generate clusters, we first need to calculate the distance between each pair of points
- We can use the Euclidean distance for this.

Example: Single linkage

Point	X	Y
A	1	1
B	2	2
C	5	5
D	6	6

dist(A,B) = sqrt((2-1)^2 + (2-1)^2) = sqrt(2)
dist(A,C) = sqrt((5-1)^2 + (5-1)^2) = 4*sqrt(2)
dist(A,D) = sqrt((6-1)^2 + (6-1)^2) = 5*sqrt(2)
dist(B,C) = sqrt((5-2)^2 + (5-2)^2) = 3*sqrt(2)
dist(B,D) = sqrt((6-2)^2 + (6-2)^2) = 4*sqrt(2)
dist(C,D) = sqrt((6-5)^2 + (6-5)^2) = sqrt(2)

	A	B	C	D
A	0	sqrt(2)	4*sqrt(2)	5*sqrt(2)
B	sqrt(2)	0	3*sqrt(2)	4*sqrt(2)
C	4*sqrt(2)	3*sqrt(2)	0	sqrt(2)
D	5*sqrt(2)	4*sqrt(2)	sqrt(2)	0

Example: Single linkage

- We can then use this proximity matrix to apply single linkage to generate clusters.
- Single linkage works by always merging the two closest clusters. Let's see how it works step by step:

1. Start with each point as a separate cluster: {A}, {B}, {C}, {D}
2. Find the two closest clusters, which are {A} and {B}. Merge them to create a new cluster: {A,B}.
3. Find the two closest clusters, which are {A,B} and {C}. Merge them to create a new cluster: {A,B,C}.
4. Find the two closest clusters, which are {A,B,C} and {D}. Merge them to create the final cluster: {A,B,C,D}.

So the final result is a single cluster containing all points: {A,B,C,D}

What's good about HC?

- ✅ No predetermined number of clusters required.
- ✅ Easy to interpret: The resulting dendrogram shows the hierarchical relationships between the clusters, making it easy to interpret and understand the structure of the data.
- ✅ Works well with small datasets: Hierarchical clustering is effective when working with small datasets where the number of observations is relatively low.
- ✅ No assumptions about data distribution: Hierarchical clustering does not make any assumptions about the distribution of the data.

What's not so good about HC?

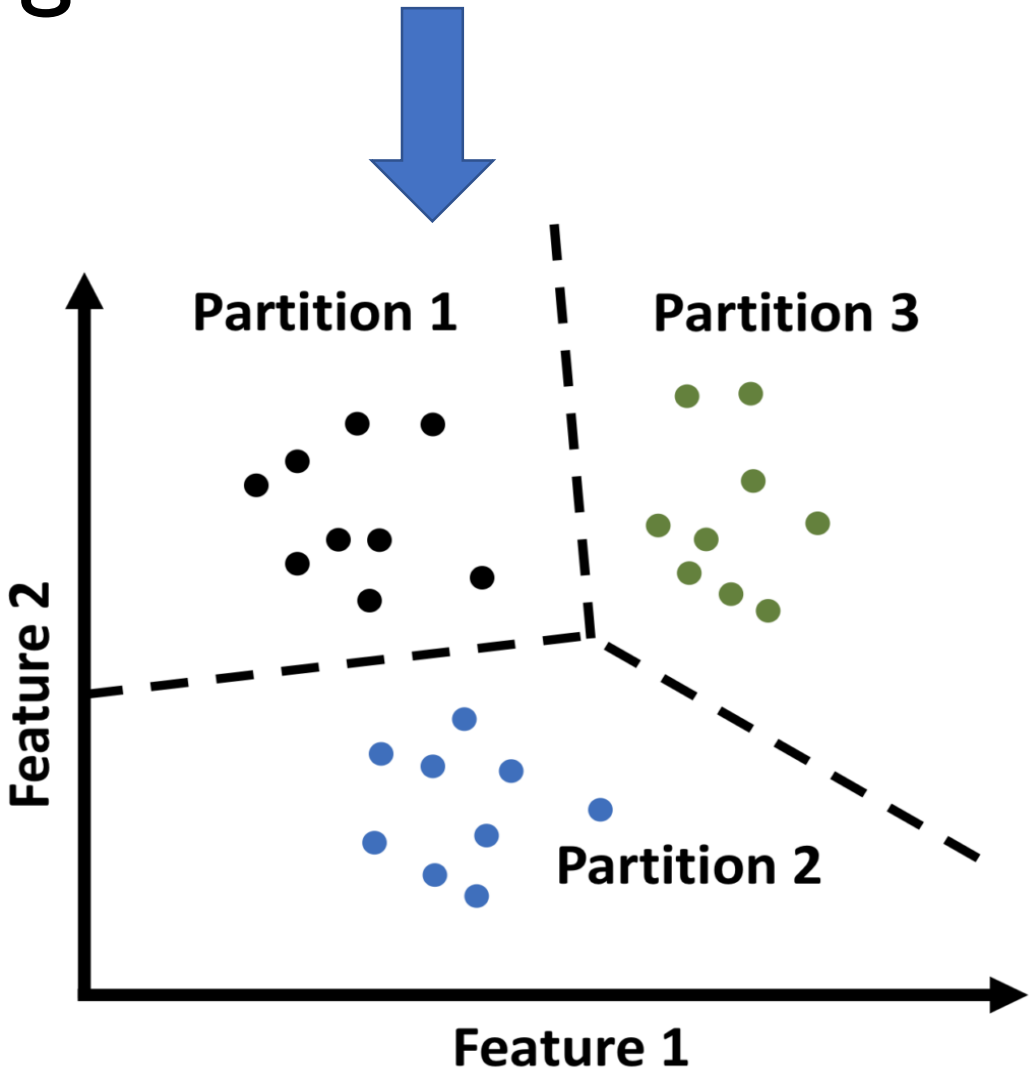
- ✕ Computationally expensive: Hierarchical clustering can be computationally expensive, especially when dealing with large datasets.
- ✕ Sensitive to noise and outliers: Hierarchical clustering is sensitive to noise and outliers, which can affect the formation of clusters and produce unreliable results.
- ✕ Lack of scalability: Hierarchical clustering is not scalable for large datasets, which can limit its practical applications in certain areas.

Conclusion HC

Useful technique for data exploration and understanding, but it may not always be the best choice for large or noisy datasets where computational efficiency and outlier robustness are important.

Partitioning-based Clustering

Partitioning



K-Means

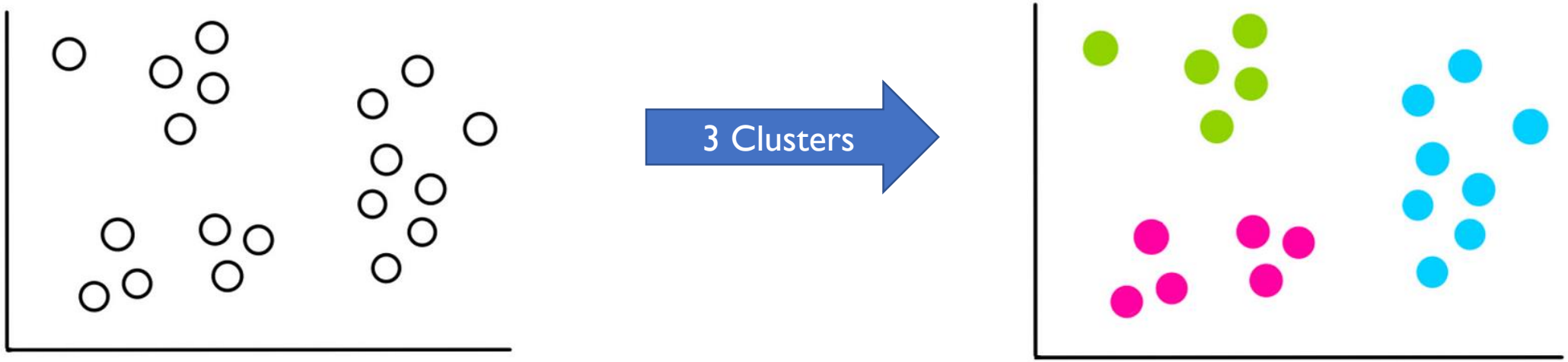
- Most commonly used unsupervised machine learning algorithm for partitioning a given data set into a set of k groups (i.e. k clusters), where k represents the number of pre-specified groups.

- In k-means clustering, each cluster is represented by its center (i.e, centroid) which corresponds to the mean of points assigned to the cluster

K-means Algorithm

- Initialization:
  - Choose the number of clusters, K, that you want to partition your data into.
  - Initialize K cluster centroids randomly. These centroids are typically chosen from the data points themselves or generated randomly within the data space.
- Assignment:
  - For each data point in your dataset, calculate the distance between the point and each of the K centroids.
  - Assign the data point to the cluster whose centroid is closest to it. This step creates K clusters.
- Update Centroids:
  - Calculate the mean (centroid) of all data points assigned to each cluster. These means become the new centroids for the clusters.
- Repeat:
  - Repeat the Assignment and Update Centroids steps until convergence. Convergence typically occurs when the centroids no longer change significantly or when a predefined number of iterations is reached.

K-Means: Example



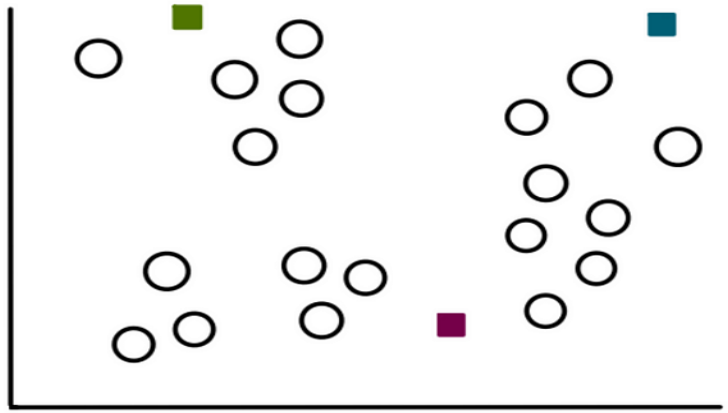
Example

Step 1: Select the Number of Clusters, k

In this case, since we assumed that there are 3 clusters, k = 3.

Step 2: Select k Points at Random

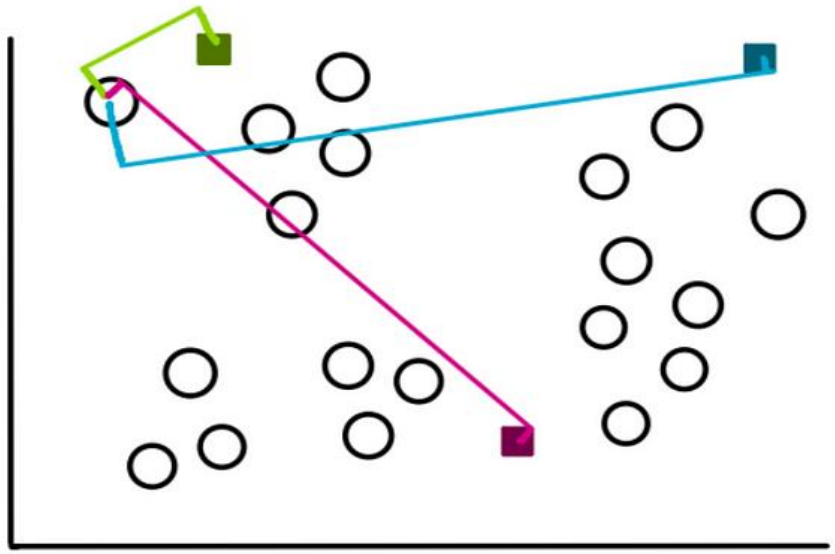
These points will now act as *centroids*, or the center, of clusters. Not necessarily our data points.



Example

Step 3: Make k Clusters

- Measure the distance from each data point to each of the 3 centroids.
- Assign the points to the cluster closest to it.



- The classical methods for distance measures are:

Euclidean distances $$d_{euc}(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

Manhattan distances $$d_{man}(x,y) = \sum_{i=1}^{n}|(x_i - y_i)|$$

Example

Step 3: Make k Clusters

Repeat this process for the rest of the points

Example

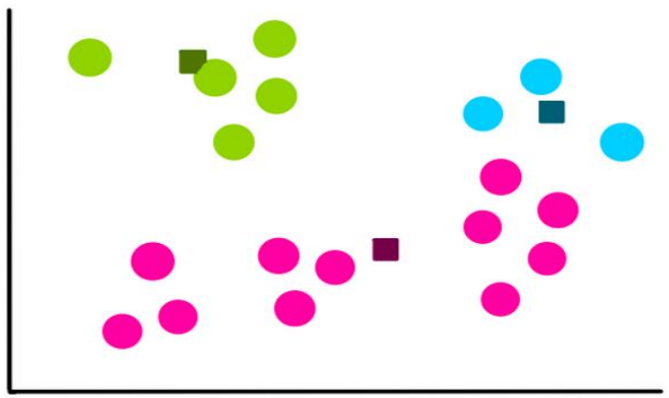
Step 4: Compute the New Centroid of Each Cluster

Find the new centroids formed by each cluster.

We use the mean to find the new centroid.

$$(x', y') = \left(\frac{1}{n}\sum_0^n x_i,\ \frac{1}{n}\sum_0^n y_i\right)$$

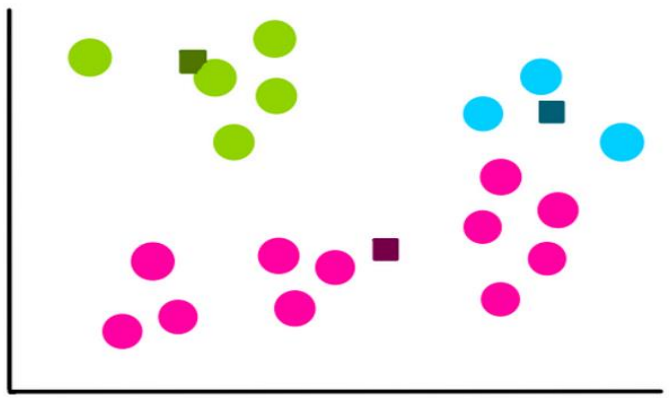
With n is the number of points in a cluster.



Example

Step 5: Assess the Quality of Each Cluster

The basic idea behind k-means clustering is defining clusters so that the within-cluster variation is minimized.



Calculate **Within-Cluster Sum of Squares (WCSS),** also called goodness of the clustering.

$$\text{WCSS} = \sum_{C_k}^{C_n} \left(\sum_{d_i \text{ in } C_i}^{d_m} distance(d_i, C_k)^2 \right)$$

Where,

C is the cluster centroids and d is the data point in each Cluster.

Example

Step 6: Repeat Steps 3–5

Iteration **2**

WCSS

Example

Step 6: Repeat Steps 3–5

Iteration **3**

WCSS

Example

Step 6: Repeat Steps 3–5

Iteration **4**

WCSS

Example

Step 6: Repeat Steps 3–5

Iteration **5**

WCSS

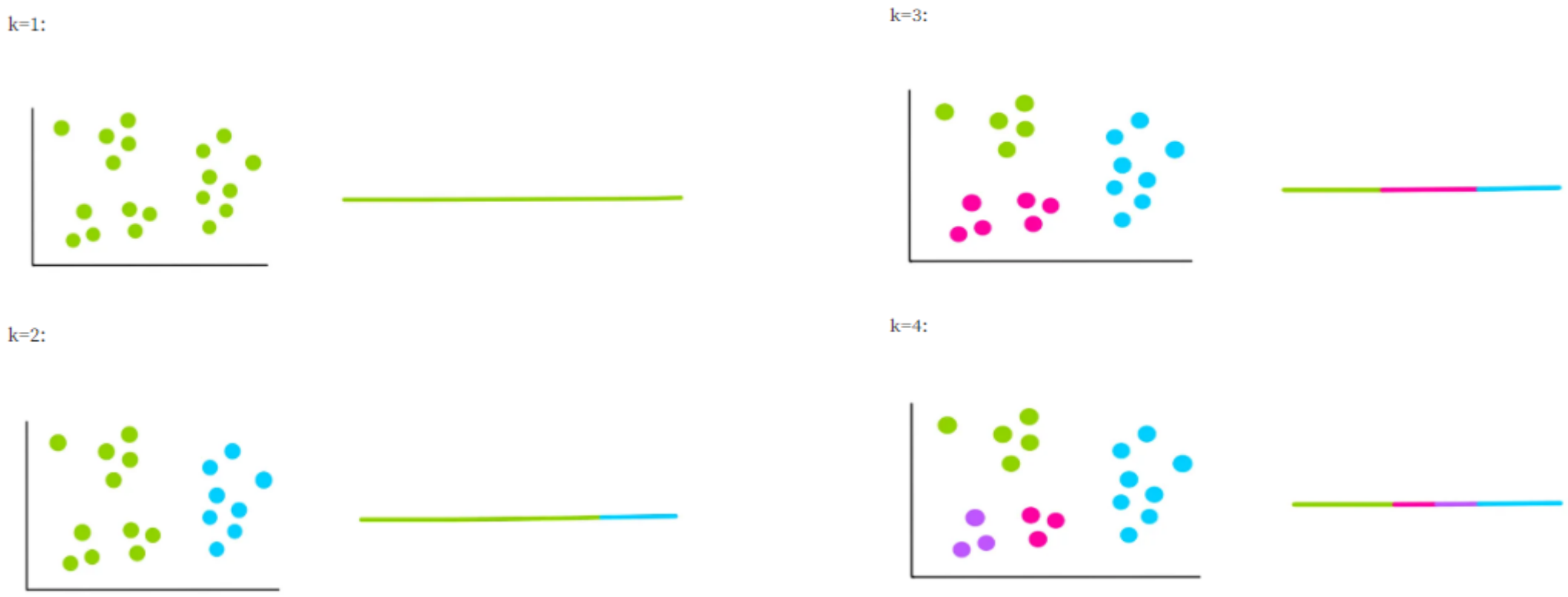
In these last two iterations, we see that:

- the clusters have not changed.
- WCSS did not change and is minimized.

This means that the algorithm
has ***converged*** and we stop the clustering process.

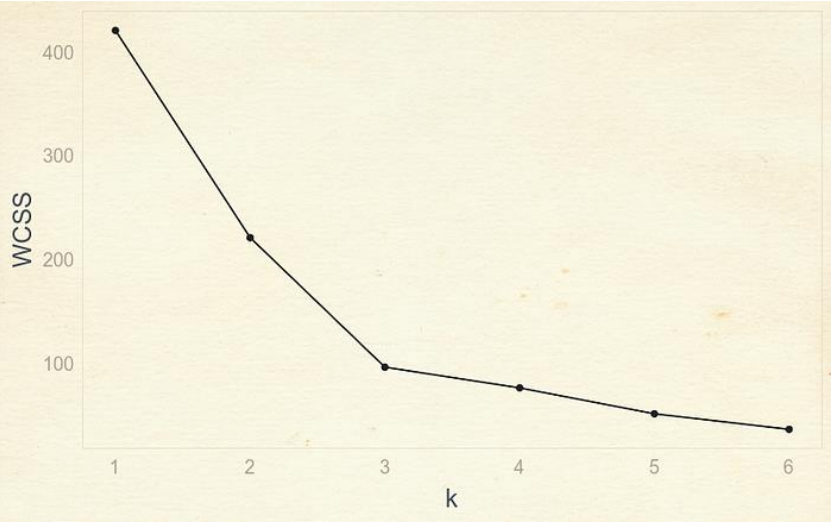
How to choose K?

WCSS is often used in conjunction with the "elbow method" to determine the optimal number of clusters (K) for your data



How to choose K? Elbow Method

- use what is called an *elbow plot* to find the best k. It plots the WCSS against the number of clusters or k.
- We can see that the graph will rapidly change at a point and thus creating an elbow shape.
- From this point, the graph moves almost parallel to the X-axis. The K value corresponding to this point is the optimal value of K or an optimal number of clusters.

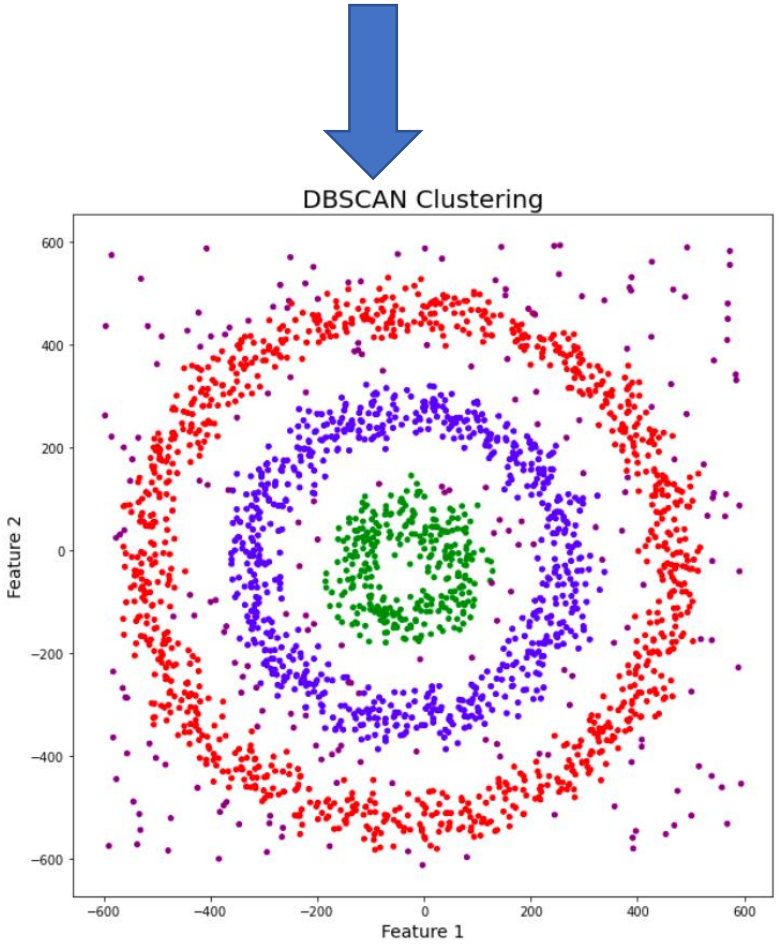


Comments on K-means

- <u>Strengths</u>
  - *Relatively efficient*: $O(tkn)$, where n is # objects, k is # clusters, and t is # iterations. Normally, $k, t << n$.
  - Very stable.
- <u>Weakness</u>
  - Applicable only when the mean is defined, then what about categorical data?
  - Need to specify k, the number of clusters, in advance
  - Unable to handle noisy data and outliers

Density-based Partitioning

Density-Based



DBScan

DBSCAN: *Density-Based Spatial Clustering of Applications with Noise*

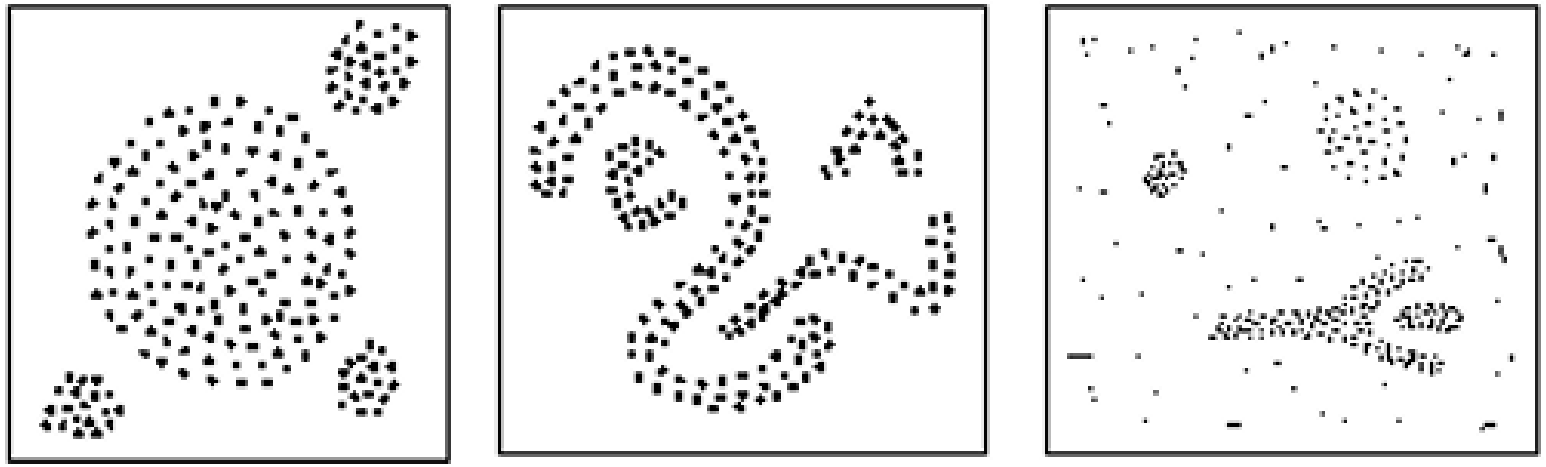
is a clustering algorithm that groups data points into clusters based on their density and identifies noise points as outliers.

Major features:

- Discover clusters of arbitrary shape
- Handle noise

DBScan

- Basic Idea: Clusters are dense regions in the data space, separated by regions of lower object density.



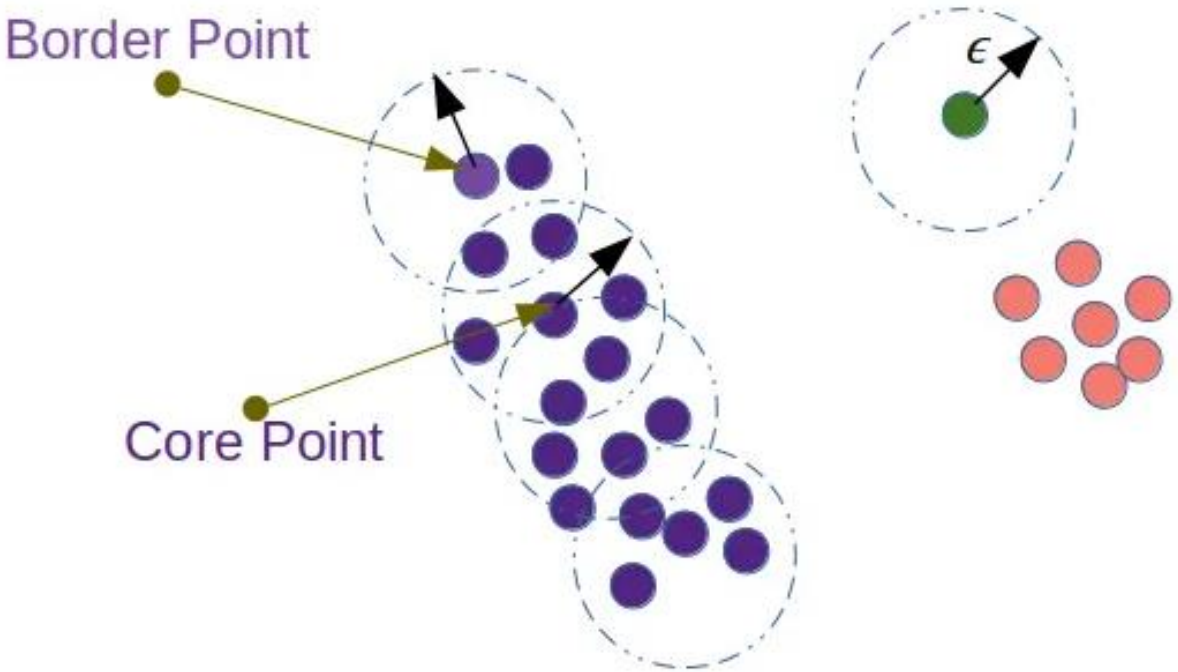
How do we measure density of a region ?

- Density at a point P: Number of points within a circle of Radius *Eps* *(ϵ)* from point P.
- Dense Region: For each point in the cluster, the circle with radius ϵ contains at least the minimum number of points (*MinPts*).

Example

A point can be classified as:

- A **core point** if it has more than a specified number of points (*MinPts*) within *Eps*.
- A **border point** if it has fewer than MinPts within Eps, but is in the neighborhood of a core point.
- A **noise point** is any point that is not a core point nor a border point.



- *Example:*
  - *MinPts = 5*
  - Radius = ϵ

Steps of DBScam

1. **Initialization**:
   - Choose two user-defined parameters: Epsilon (*Eps*) and *MinPts*.
   - Epsilon (*Eps*) defines the maximum distance for points to be considered neighbors.
   - *MinPts* specifies the minimum number of points required to form a core point.
2. **Core Point Identification**:
   - For each data point in the dataset, calculate the distance to all other data points.
   - Count the number of data points within the *Eps* distance of the current point.
   - If the count is greater than or equal to *MinPts*, mark the point as a core point.

Steps of DBScam

3. **Cluster Expansion**:
   - Start with an unprocessed core point and create a new cluster.
   - Add the core point to the cluster and identify its *Eps*-neighborhood.
   - For each point in the *Eps*-neighborhood:
     - If the point is also a core point, add its Eps-neighborhood to the cluster.
     - If the point is not already assigned to a cluster, add it to the cluster.
   - Repeat this process until no more points can be added to the cluster.
4. **Repeat for All Core Points**:
   - Repeat the cluster expansion process for all unprocessed core points.
   - This step ensures that all core points and their directly and indirectly reachable points are assigned to clusters.

Steps of DBScam

5. **Border Points**:
   - Assign border points (points within the *Eps*-neighborhood of a core point but not core themselves) to their respective clusters.
6. **Noise Points**:
   - Any unvisited points that are not core points or border points are classified as noise points (outliers) and are not assigned to any cluster.

- See simulation here

Comments on DBScan

- DBSCAN is particularly useful for discovering clusters of arbitrary shapes and handling noisy data. It does not require specifying the number of clusters in advance and can uncover clusters with varying densities.

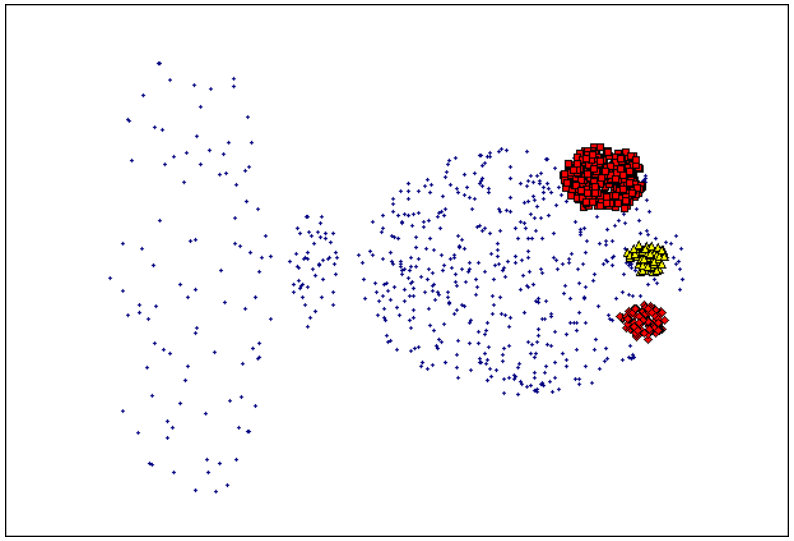
Time and Space Complexity:

- Time Complexity: range from O(n log n) to O(n^2), where 'n' is the number of data points in the dataset.

- Space Complexity: O (n).

Comments on DBScan

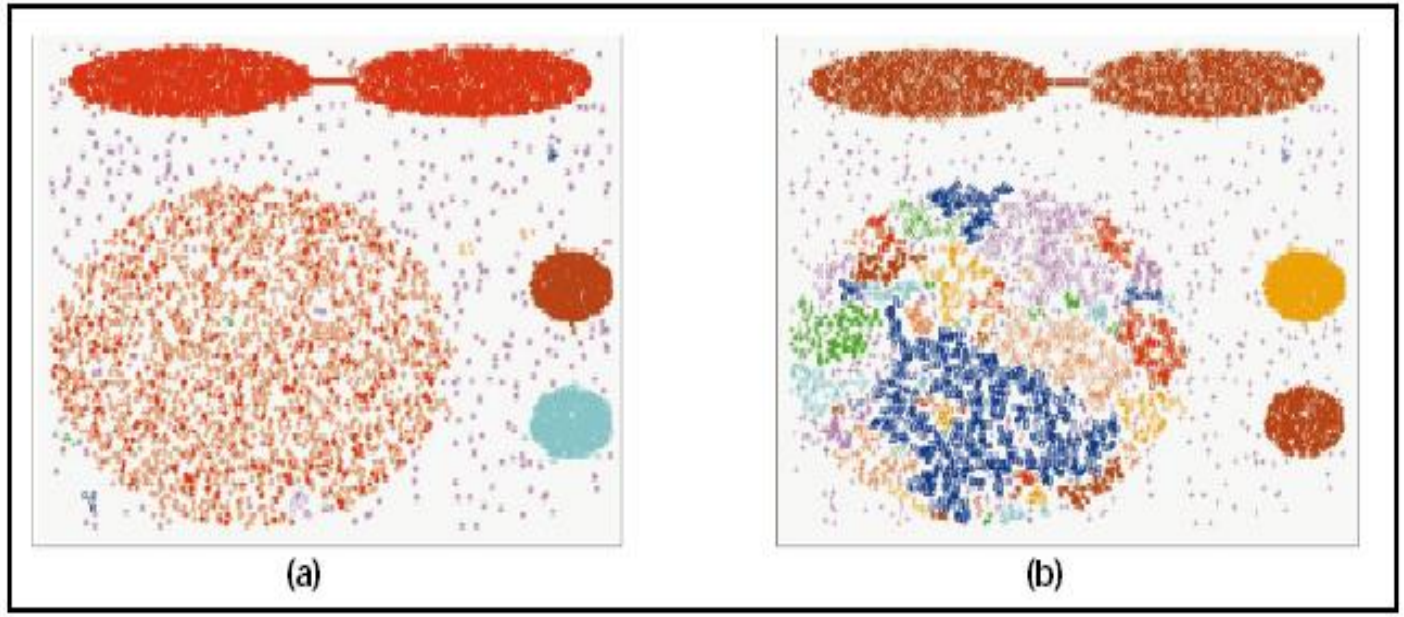
If the database has data points that form clusters of <u>varying density</u>, then DBSCAN fails to cluster the data points well, since the clustering depends on ϵ and *MinPts* parameters, they cannot be chosen separately for all clusters.



Comments on DBScan

- If the data and features are not so well understood then, <u>setting up ϵ and MinPts</u> could be tricky and, may need comparisons for several iterations with different values of ϵ and MinPts.

Figure 8. DBScan results for DS1 with MinPts at 4 and Eps at (a) 0.5 and (b) 0.4.

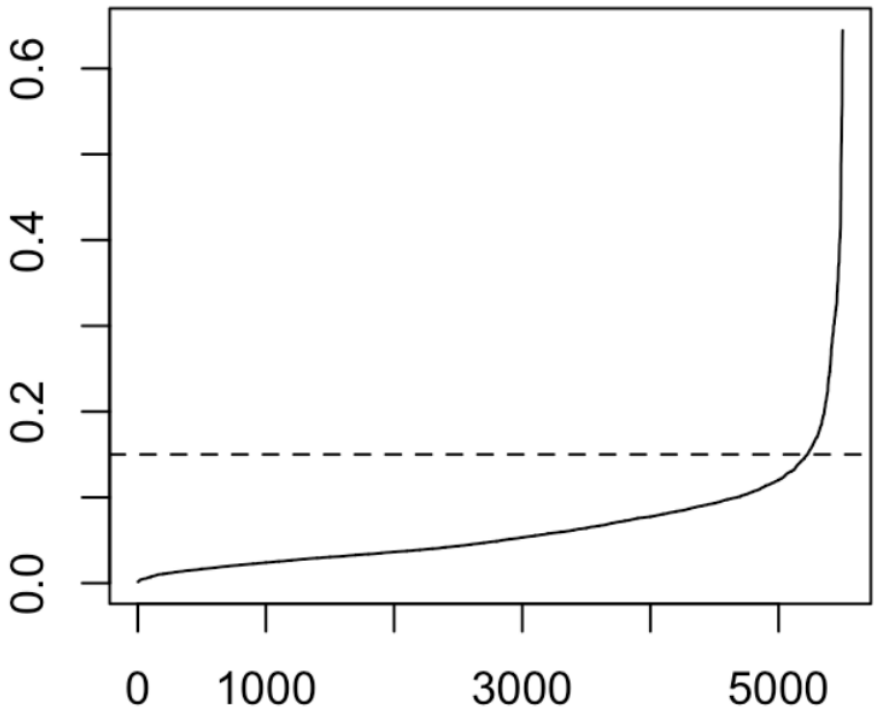


How to find the required parameters for DBSCAN clustering?

- The minPts **should be 4** for two-dimensional dataset.
- For multidimensional dataset, minPts should be **2 * number of dimensions**. For example, if the dataset has 6 features, minPts = 12.

How to find the required parameters for DBSCAN clustering?

- The idea is to calculate, the average of the distances of every point to its k nearest neighbors.
- The value of k will be specified by the user and corresponds to *MinPts*.
- Next, these k-distances are plotted in ascending order. The aim is to determine the "knee", which corresponds to the optimal *epsilon* parameter. A knee corresponds to a threshold where a sharp change occurs along the k-distance curve.



DBScan vs K-Means

K-means Clustering	DBScan Clustering
K-means clustering is sensitive to the number of clusters specified	No need to specify the number of clusters
More efficient for large datasets	Can not efficiently handle high dimensional datasets.
Does not work well with outliers and noisy datasets.	Efficiently handles outliers and noisy datasets.
It requires one parameter : Number of clusters (K)	It requires two parameters : Radius and Minimum Points.
Varying densities of the data points doesn't affect results	Does not work very well for sparse datasets or for data points with varying density.
Low complexity	Medium/high complexity